

JS - AI Capability Exercise

Participant Guide — Build & Grow Your AI Workflow

A hands-on exercise every developer in the competency completes to strengthen and show how they work with AI. You'll get a feedback report and a personalized growth path — this is for development, not a graded test.

Contents

- 1. What This Is
- 2. Who Takes Part
- 3. Time and Effort
- 4. What You Get Out of It
- 5. How the Exercise is Structured
- Part A: AI Workflow Foundation
- Part B: Full-Stack Mini Project
- Submission Templates
- Tool-Specific Expectations
- What Counts as Complete
- How to Take Part
- What Good Looks Like
- Your Growth Path
- Summary

1. What This Is

This is a hands-on capability exercise to help you develop — and make visible — how you use AI tools effectively, responsibly, and practically across the software development lifecycle. Everyone in the competency takes part; it is a shared part of how we build AI capability, not something a few people are singled out for.

It is not a graded test. You will build a small full-stack project and show your thinking, and in return you get a feedback report and a clear sense of what to grow next. What matters is not only whether the final app works, but how you used AI for requirement analysis, planning, implementation, testing, debugging, code review, documentation, and reflection. Making your thinking visible is the point.

2. Who Takes Part

All developers in the competency, from SE to TL level, work with stacks such as React, Node.js, or full-stack combinations. It is common across roles and stacks, though the project option you choose may emphasize either backend or frontend depth. Because everyone does it, there is a shared baseline and no one is measured against a different bar than their peers.

3. Time and Effort

The exercise is self-paced and meant to be completed within one week. You may work in any order; there is no required day-wise plan. Share your work by the agreed date.

Expected effort: the mandatory Core of the project is scoped for roughly 8–12 focused hours. The rest of the week goes into the lifecycle artifacts — requirement analysis, prompt history, testing and debugging notes, reflection — which are the main thing the feedback looks at. Do not expand the Core application at the expense of these artifacts.

4. What You Get Out of It

You receive a feedback report: your strengths, your growth areas, and concrete next steps for developing your AI workflow. The report also gives a sense of where you currently sit in the AI capability framework and what would move you forward — think of it as a snapshot and a direction, not a grade. Your competency owner uses this as one input into your normal growth track.

Feedback focuses on areas like requirement analysis, prompting and context-setting, tool workflow, full-stack design, code quality, database design, testing depth, debugging, code review, documentation, ownership, and responsible AI judgment — so you leave with a clear picture of what to practice next.

5. How the Exercise is Structured

It has three parts:

Part	Focus	Emphasis
Part A	AI Workflow Foundation	20%
Part B	Full-Stack Mini Project (Core + optional Stretch)	60%
Part C	Submission and Reflection	20%

Part C is completed through the participation form (see “How to Take Part”), where you share your repository and answer a few questions about your work. The percentages show where to put your effort — they are not exam weights.

Part A: AI Workflow Foundation

Objective

Show that you understand how AI should be used in a practical engineering workflow — thoughtfully, not as a simple code-generation shortcut.

Expected Submission

Submit a document named `tool-workflow.md` covering:

1. Primary AI tool used.
2. How you provide project context to the tool.
3. How you use AI for requirement analysis.
4. How you use AI for planning and design.
5. How you use AI for code generation.
6. How you validate AI-generated code.
7. How you use AI for testing.
8. How you use AI for debugging.
9. How you use AI for code review.
10. What information you avoid sharing unnecessarily with AI tools.
11. How you would reuse this workflow in a real project.

Part B: Full-Stack Mini Project

Objective

Demonstrate practical AI-assisted delivery through a realistic full-stack assignment. Each project is split into a mandatory Core and an optional Stretch tier. Core is a small, buildable application that serves as a vehicle for showing your AI workflow. Stretch is optional and shows more advanced practice.

A clean, well-documented Core alone is a strong result. Both options are looked at the same way; the difference is the depth of evidence you show.

Common Technical Requirements

Whichever option you choose, every submission must include:

1. Frontend application
2. Backend API
3. Database persistence
4. Database setup or migration scripts
5. Seed or sample data
6. Input validation
7. Error handling
8. One working search or filter capability (Core); more in Stretch
9. At least one meaningful test tier (Core); more in Stretch
10. README setup instructions
11. Full prompt history
12. All planning, design, testing, debugging, review, reflection, and PR artifacts in the repository structure

The full set of lifecycle artifacts is required regardless of option. Only the application surface area is small — the artifacts are the point.

Database Requirement

A database is mandatory. Acceptable options include PostgreSQL, MySQL, MongoDB, SQLite, H2, or any framework-supported local database. Provide your database choice, setup instructions, a schema/migration/initialization script, seed data, an environment variable example (if applicable), and steps to run locally.

Authentication

Authentication is optional — the focus is AI-assisted engineering across the lifecycle, not auth. If implemented well it counts as Stretch evidence (login/logout, JWT or session auth, role-based access, protected routes, API authorization).

Project: Support Ticket Management System

Business Context

A small application for managing support tickets. Internal users create, update, comment on, search, and progress tickets through a defined lifecycle.

Core (Mandatory)

Entities

```
User      (seeded only — no user-management UI required)
- id, name, email, role

Ticket
- id, title, description, priority, status,
  assignedTo, createdBy, createdAt, updatedAt

Comment
- id, ticketId, message, createdBy, createdAt
```

Features

1. Create a ticket.
2. List tickets.
3. View ticket details.
4. Update ticket fields (title, description, priority, assignee).
5. Change ticket status through the enforced state machine.
6. Add comments to a ticket.
7. Keyword search and filter by status.
8. Persist all data; data survives restart.
9. Validate required fields; reject invalid input at the backend.
10. Show meaningful error states in the UI.

Status state machine *(the signature judgment piece — kept in Core)*

```
Open      -> In Progress
In Progress -> Resolved
Resolved  -> Closed
Open      -> Cancelled
In Progress -> Cancelled
```

Invalid transitions must be rejected by the backend and handled clearly in the frontend. This is deliberately the hardest part of Core because it is where engineering judgment shows.

Mandatory test tier: integration tests that prove the state-machine rules — valid transitions succeed, invalid transitions are rejected.

Stretch (Optional — evidence toward C1.1)

- Third entity or richer data model
- Full user CRUD and role management
- Authentication, protected routes, API authorization checks
- Filter by priority and assignee; sorting; pagination
- Additional test tiers: unit tests and edge-case/failure tests
- API documentation (Swagger / OpenAPI)
- Docker setup, CI workflow
- Reusable prompt templates, rules, or specs (persistent project context)

Core Acceptance Criteria

1. A user can create a ticket via the UI.
2. A user can view all tickets from the database.
3. A user can open a ticket detail view.
4. A user can update ticket fields and reassign.
5. A user can add comments.
6. Status changes only through valid transitions; invalid ones are rejected.
7. Keyword search and status filter work.
8. Data remains available after restart.
9. Backend validation prevents invalid records.
10. No secrets committed to the repo.
11. State-machine integration tests pass.

Tool-Specific Expectations

Kiro

Submit `tool-specific/kiro-specs/` with `requirements.md`, `design.md`, `tasks.md`. Show that you reviewed and improved the generated spec, that your implementation followed it, that you updated it when needed, and that you used Kiro as a spec-driven tool rather than only a code generator.

Cursor

Submit `tool-specific/cursor-workflow/` with `project-context.md`, `spec.md`, `tasks.md`, `acceptance-criteria.md`, and `cursor-rules-or-instructions.md`. Show persistent project context, spec-driven development via documents and rules, iteration beyond first output, and traceability from spec to implementation.

Claude or other approved tools

Submit `tool-specific/other-tool-workflow/` with `project-context.md`, `spec.md`, `tasks.md`, `acceptance-criteria.md`, and `tool-usage-notes.md`. Show enough project context, structured planning before implementation, validation of AI suggestions, and traceability between requirement, design, code, and tests.

Note: *you are not disadvantaged by your choice of tool. Stronger work shows reusable, persistent project context and standards enforced through your tool of choice — not a specific product's artifacts.*

What Counts as Complete

For your feedback to be useful, your submission should include a working frontend, working backend, database persistence, database setup or migration files, seed data, README setup instructions, basic tests, prompt history, requirement analysis, design notes, reflection, and a PR description. If pieces are missing you will still get feedback — it just won't reflect the full picture, and the growth pointers will focus on filling those gaps.

How to Take Part

Once your project is ready, you share it through a single online form. There is no review call to book and nothing to host or deploy.

What you share

1. A link to your Git repository (make sure it is accessible to the competency team).
2. Your selected project option and primary AI tool.
3. Short written answers to a few questions in the form.

About the questions

The questions ask you to explain your work in your own words — your requirement understanding, how you used AI across the lifecycle, key code and design decisions, your testing and debugging approach, and what you'd improve. A few ask you to point to specific places in your repository (for example, the commit where you fixed an AI mistake), so keep your commit history and prompt history organized as you go.

Be specific and honest. This is for your own development, so there's no value in generic or inflated answers — they just produce less useful feedback. The clearer the picture you give of how you actually worked, the more useful your growth pointers will be.

Follow-up and coaching

Your work is reviewed and a feedback report is shared with you. Sometimes a mentor or your competency owner may follow up to talk through your project and coach you on next steps — a short, supportive conversation, not a re-examination. This is part of how the competency helps you grow, not a hurdle.

What Good Looks Like

The feedback weighs how you used AI across the lifecycle and how well you understand and own your solution — not just whether the app runs. At a high level:

Strong work usually shows

- Clear understanding of the problem before coding, with good requirement breakdown and acceptance criteria
- Well-documented AI prompts with iteration — context-setting, refinement, and correction of wrong suggestions
- Clean, maintainable code and a working database setup that runs from the README
- Meaningful tests and clear evidence of debugging and review
- An honest reflection, the ability to explain trade-offs, and reusable workflow artifacts

Weaker work usually shows

- Direct copy-paste from AI with little understanding; missing requirement analysis
- Missing or shallow prompt history; no clear design
- Broken setup instructions or no database persistence
- Superficial tests; no debugging or review evidence
- Generic documentation; inability to explain the generated code; no ownership of the result

In short: the journey and your understanding matter as much as the final code. Make your thinking visible.

Your Growth Path

Whatever your starting point, the feedback comes with a direction for what to develop next:

- **Building the basics:** focus on requirement analysis, providing context to your AI tool, and getting to working, tested output. A follow-up exercise or learning path may be suggested.
- **Solid and growing:** you produce working AI-assisted output; next is deepening tests, review quality, and debugging maturity.
- **Strong across the lifecycle:** you use AI well end-to-end and can explain your decisions; next is advanced testing and reusable workflows.
- **Ready to lead:** mature workflow with reusable prompts, rules, or specs — a natural point to start sharing practices and mentoring others.

Your competency owner uses the report and this direction within your normal growth conversations — it informs where you are in the capability framework rather than stamping a grade on you.

Summary

You'll complete a self-paced, one-week AI-assisted full-stack exercise using an approved AI tool (Cursor, Kiro, Claude, or equivalent). Choose one project option — Backend-heavy (Support Ticket Management System) or Frontend-heavy (AI Learning Dashboard / Project Tracker) — and build the mandatory Core; Stretch is optional and shows more advanced practice.

Your project should include a frontend, a backend API, database persistence, setup/migration files, seed data, validation, error handling, search/filter, tests, README instructions, AI prompt/workflow history, a reflection, and a PR description. Authentication is optional. You may use React, Node.js, Java, Python, or a full-stack combination.

Use AI thoughtfully and make your usage visible — how you set context, planned, generated and refined code, tested, debugged, reviewed, and documented. When you're ready, share your repository link and answers through the form. You'll get a feedback report with your strengths, growth areas, and next steps, and possibly a short coaching conversation. Everyone in the competency does this — it's a shared step in building how we all work with AI.